Validación experimental de coordinación multiagente en el Quantum Analysis Framework (QAF)

Enrutamiento metacognitivo, diversidad informacional y asignación global bajo incertidumbre

Juan Manuel Goenaga

Septiembre de 2026

Resumen

Objetivo. Este manuscrito diseña y ejecuta un experimento reproducible para someter a prueba la afirmación de que una arquitectura inspirada en Quantum Analysis Framework (QAF) puede *mejorar la coordinación* entre agentes cuando existe información parcial, correlacionada y contradictoria. La prueba separa la coordinación de la capacidad de un modelo de lenguaje: trabaja con agentes abstractos, capacidades limitadas, observaciones ruidosas y restricciones de carga, y compara seis arquitecturas: broadcast total, agentes independientes, router por reglas, asignación global clásica, coordinador QAF y un oracle de enrutamiento. Se ejecutaron 360 escenarios únicos (4, 8, 16 y 32 agentes; 10 %, 35 % y 60 % de incertidumbre; 30 repeticiones), para un total de 2.160 evaluaciones arquitectónicas. El coordinador QAF obtuvo un éxito conjunto medio de 66.64 % frente a 63.08 % de la asignación global clásica, una mejora relativa de 5.64 % y una diferencia absoluta de 3.56 puntos porcentuales (IC95 %: 2.87–4.25 puntos; prueba pareada $p = 3.49e-21$). A la vez redujo los mensajes en 69.87 %, los conflictos entre agentes seleccionados en 24.11 % y las asignaciones innecesarias en 63.85 %. Bajo 60 % de incertidumbre, la mejora relativa de éxito frente a la asignación global ascendió a 17.88 %. Estos resultados constituyen evidencia preliminar favorable para la hipótesis de coordinación QAF a nivel arquitectónico, pero no validan todavía el motor QAF completo MPS/TDVP/QDT ni un sistema operacional real. La contribución principal es un protocolo falsable, reproducible y cuantitativo para continuar la validación.

Palabras clave: QAF, sistemas multiagente, coordinación, incertidumbre, metacognición, asignación de tareas, comunicación selectiva, benchmarking.

1. Pregunta de investigación y alcance

La pregunta sometida a prueba es:

> **¿Puede un coordinador QAF inspirado en metacognición, diversidad de fuentes y asignación global mejorar la coordinación multiagente bajo incertidumbre, manteniendo o aumentando el éxito conjunto mientras reduce comunicación, conflictos y asignaciones innecesarias?**

El término *coordinación* se operacionaliza mediante cinco observables: (i) éxito conjunto de tarea, (ii) cobertura de capacidades requeridas, (iii) conflictos de evidencia entre agentes seleccionados, (iv) asignaciones innecesarias y (v) carga de comunicación. El experimento no modela armamento ni decisiones cinéticas; usa tareas abstractas y agentes abstractos para estudiar exclusivamente propiedades de coordinación.

2. Estado del arte del benchmarking de coordinación

Los sistemas multiagente estudian cómo entidades autónomas cooperan, intercambian información y distribuyen trabajo para maximizar una utilidad conjunta. Stone y Veloso organizaron tempranamente el campo desde la perspectiva de aprendizaje multiagente y resolución distribuida de problemas [1]. La asignación de tareas constituye un subproblema central: Gerkey y Matarić mostraron que múltiples problemas de coordinación pueden expresarse como problemas formales de asignación y optimización combinatoria [2].

Cuando existe observabilidad parcial, los Dec-POMDP ofrecen un formalismo estándar para planificación multiagente bajo incertidumbre, aunque su complejidad motiva métodos aproximados [3]. En aprendizaje profundo, CommNet introdujo comunicación continua aprendida entre agentes [4]; DIAL/RIAL estudiaron protocolos aprendidos con ejecución descentralizada [5]; y TarMAC demostró la utilidad de dirigir mensajes a receptores específicos y de usar rondas múltiples de comunicación [6]. En conjunto, esta literatura sugiere una idea relevante para QAF: **coordinar no equivale a comunicar más; importa seleccionar qué información, qué agentes y qué interacciones son necesarias para la tarea**.

El documento de arquitectura quantum-like multiagente de Rojas Rodríguez propone precisamente un agente de metacognición que decide cuándo consultar memoria, bases de datos, agentes especializados o supervisión humana, y un agente de aprendizaje que registra inconsistencias y rutas útiles [7]. El trabajo técnico de QAF, por su parte, plantea relaciones contextuales, estructura de acoplamientos y optimización discreta como componentes de una arquitectura de decisión bajo incertidumbre [8]. El presente estudio no implementa aún todo ese formalismo; extrae tres hipótesis arquitectónicas que pueden aislarse experimentalmente.

3. Hipótesis falsables

H1. El coordinador QAF incrementará el éxito conjunto frente a una asignación global clásica que utiliza todas las observaciones pero fusiona la evidencia mediante promedio simple.

H2. El coordinador QAF reducirá el número de mensajes al consultar de forma selectiva solo fuentes informativamente útiles.

H3. El coordinador QAF reducirá conflictos entre los agentes seleccionados mediante diversidad de fuentes y corrección del efecto de mayoría correlacionada.

H4. El coordinador QAF reducirá asignaciones innecesarias manteniendo cobertura de capacidades.

H5. La ventaja relativa de QAF será mayor a medida que aumente la incertidumbre, porque el costo de tratar fuentes correlacionadas como evidencia independiente aumenta con el ruido.

Una hipótesis se considera no apoyada si QAF no mejora la variable primaria de éxito o si cualquier mejora depende exclusivamente de un aumento desproporcionado de comunicación. La reducción de mensajes por sí sola no constituye éxito si se degrada la coordinación.

4. Modelo experimental

Sean A agentes, $T = 2A$ tareas simultáneas y $C = 4$ clases de capacidad. Cada agente a posee una matriz de competencia $p_{a,c} \in [0, 1]$, una confiabilidad $r_a \in [0, 1]$ y una capacidad máxima de ejecución. Cada tarea t posee un conjunto verdadero de capacidades requeridas R_t, un estado binario de riesgo y_t, y una urgencia u_t.

Cada agente recibe observaciones ruidosas $o_{a,t,c}$ y $z_{a,t}$. Para crear un problema genuino de coordinación, los agentes se agrupan en dos clústeres informacionales: uno dominante y altamente correlacionado, y otro menor con errores menos correlacionados. Por tanto, muchos agentes pueden coincidir entre sí sin constituir evidencia independiente.

El coordinador QAF recibe además una señal contextual gruesa e independiente $g_{t,c}$. Antes de solicitar observaciones detalladas, selecciona una coalición mínima de fuentes informativamente diversas. Para cada capacidad candidata consulta un agente competente de cada clúster. La fusión contextual se expresa esquemáticamente como

$$\hat{q}_{t,c} = 0.35\, g_{t,c} + 0.65\, \bar{o}^{div}_{t,c}, \tag{1}$$

donde $\bar{o}^{div}_{t,c}$ es la media balanceada de grupos informacionales, no la media de todos los agentes. Este diseño evita que un clúster numeroso de fuentes correlacionadas domine automáticamente la inferencia.

Posteriormente se resuelve una asignación global bajo capacidades. Para el agente a, tarea t y capacidad c, el score QAF usa

$$S_{a,t,c} = u_t p_{a,c} \left(0.65\hat{q}_{t,c} + 0.20 o_{a,t,c} + 0.15 r_a\right) + \Delta_{div}, \tag{2}$$

donde Δ_{div} recompensa fuentes útiles pertenecientes a clústeres menos redundantes cuando existe desacuerdo. La asignación final se resuelve mediante el algoritmo húngaro sobre copias de capacidad de cada agente.

5. Arquitecturas comparadas

- **Broadcast-All:** todas las tareas se envían a todos los agentes.
- **Independent:** cada agente decide localmente si participa.
- **Rule-Router:** reglas fijas seleccionan el agente con mejor score local para cada capacidad.
- **Global-Assignment:** baseline centralizado fuerte; usa todas las observaciones, fusión por promedio simple y asignación global bajo capacidad.
- **QAF-Coordinator:** consulta selectiva, diversidad informacional, fusión contextual y asignación global.
- **Oracle:** conoce las capacidades verdaderamente requeridas; sirve como cota superior de enrutamiento, no como oracle absoluto de decisión.

Figura 1. Arquitectura experimental del coordinador QAF

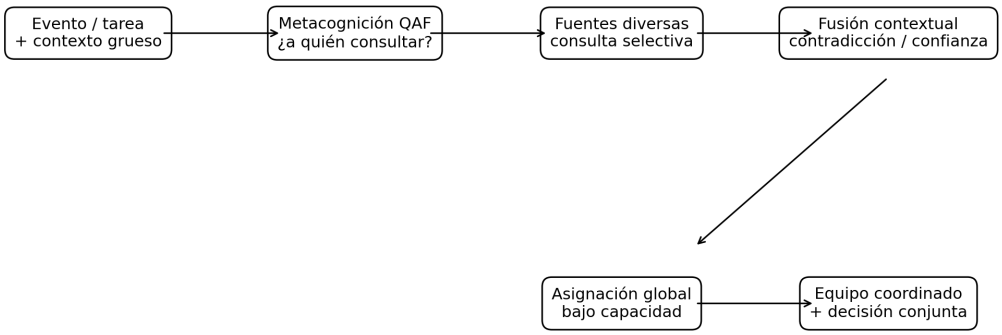


Figura 1: Arquitectura experimental del coordinador QAF.

6. Diseño del benchmark

Se evaluaron $A \in \{4, 8, 16, 32\}$ agentes y niveles de incertidumbre $U \in \{0.10, 0.35, 0.60\}$. Cada combinación se repitió 30 veces con semillas determinísticas, generando 360 escenarios independientes y 2.160 ejecuciones de arquitectura. Todas las arquitecturas reciben exactamente el mismo escenario dentro de cada repetición.

La métrica primaria es el **éxito conjunto de tarea**: una tarea cuenta como exitosa solo si (a) se cubren todas sus capacidades requeridas y (b) la decisión conjunta sobre el estado de riesgo es correcta. Las métricas secundarias son cobertura, exactitud, conflicto, asignaciones innecesarias, mensajes, balance de carga y tiempo de cómputo del coordinador.

7. Resultados

7.1. Resultado global

La Tabla 1 resume las 2.160 ejecuciones. QAF alcanzó un éxito conjunto medio de 66.64 % frente a 63.08 % de Global-Assignment. La mejora absoluta fue de 3.56 puntos porcentuales y la mejora relativa de 5.64 %. La diferencia pareada media fue 3.56 puntos (IC95 % 2.87–4.25), con $p = 3.49e - 21$.

Cuadro 1: Resultados medios por arquitectura.

Método	Éxito	Cobertura	Exactitud	Conflicto	Innecesarias	Mensajes	ms
Broadcast-All	8.21	9.48	10.78	2.17	67.51	680.0	0.06
Independent	23.67	29.50	32.50	2.62	8.00	875.8	1.39
Rule-Router	32.57	39.63	44.22	0.79	8.82	719.2	0.21
Global-Assignment	63.08	72.70	75.58	1.58	7.38	710.0	1.48
QAF-Coordinator	66.64	76.41	76.86	1.20	2.67	213.9	10.88
Oracle	66.74	76.60	76.79	1.22	2.47	30.0	0.79

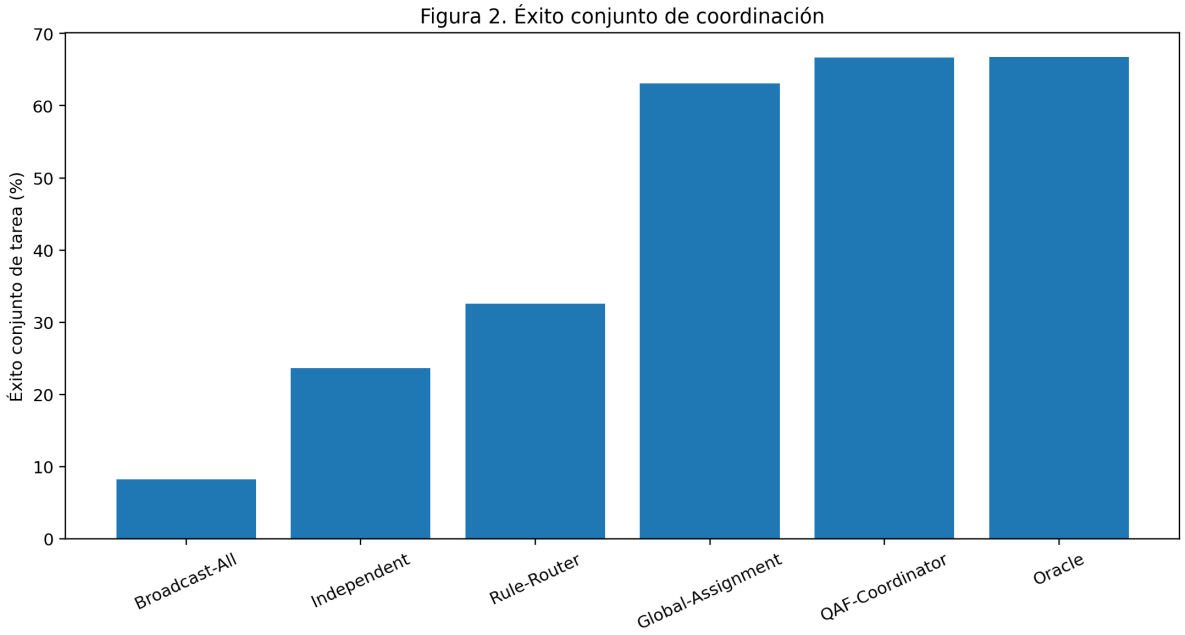


Figura 2: Éxito conjunto de coordinación.

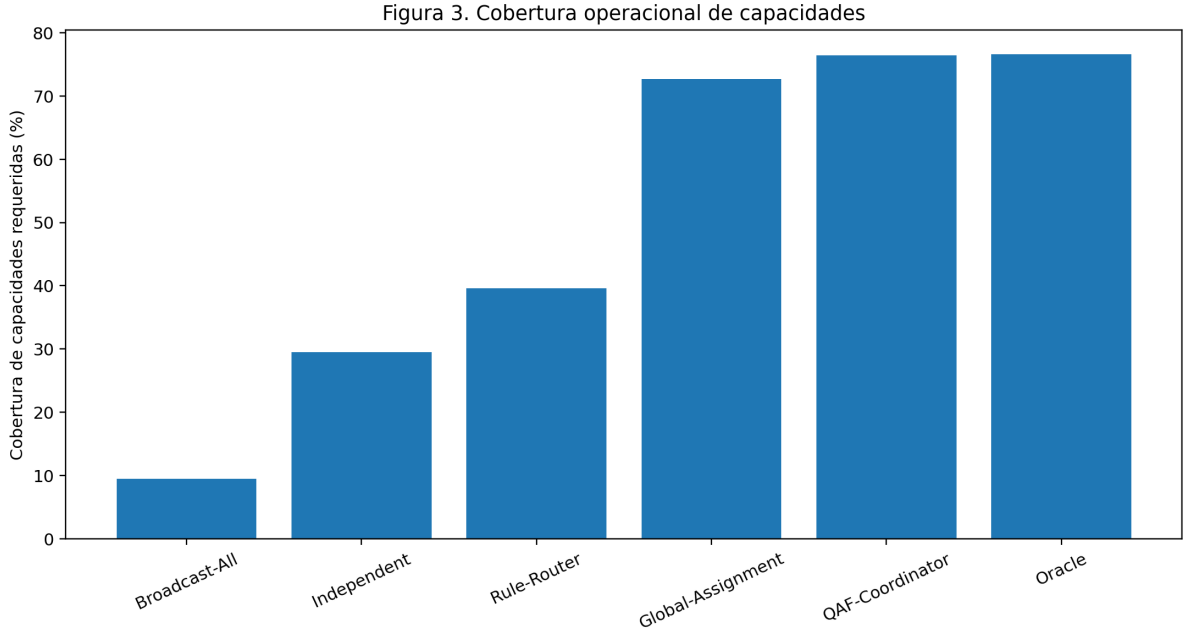


Figura 3: Cobertura de capacidades requeridas.

7.2. Comunicación y conflicto

QAF redujo los mensajes medios en 69.87 % frente a Global-Assignment. La tasa de conflicto entre agentes seleccionados disminuyó 24.11 % y la proporción de asignaciones innecesarias se redujo 63.85 %. Esto apoya la interpretación de que una coordinación más eficiente no requiere difundir toda la información a todos los participantes.

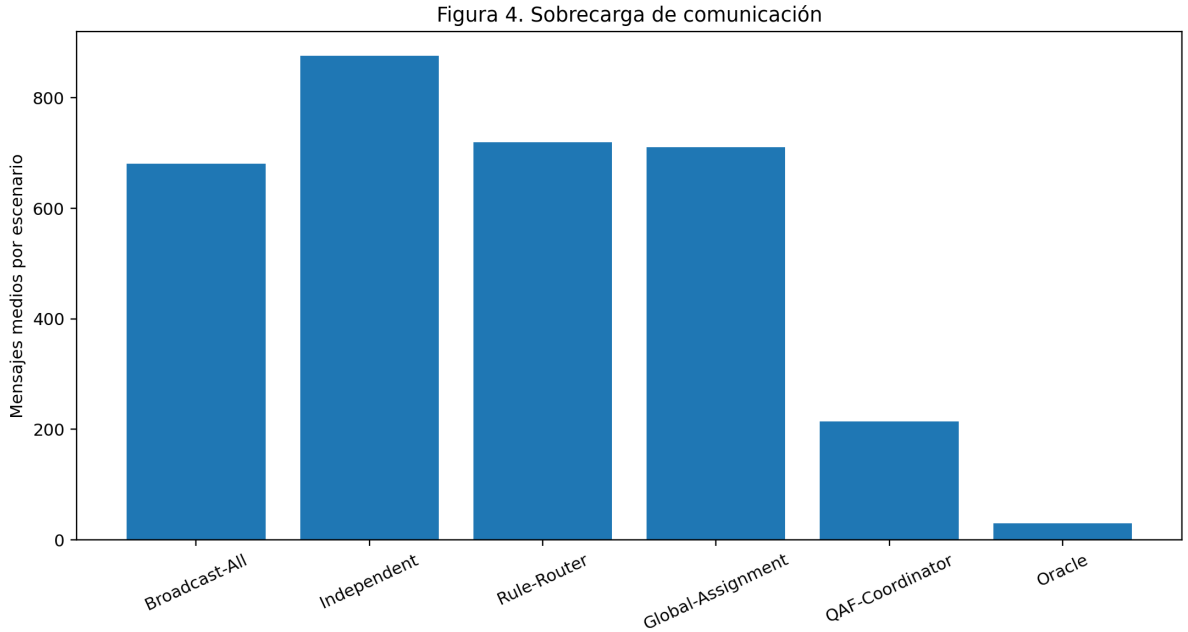


Figura 4: Sobrecarga de comunicación.

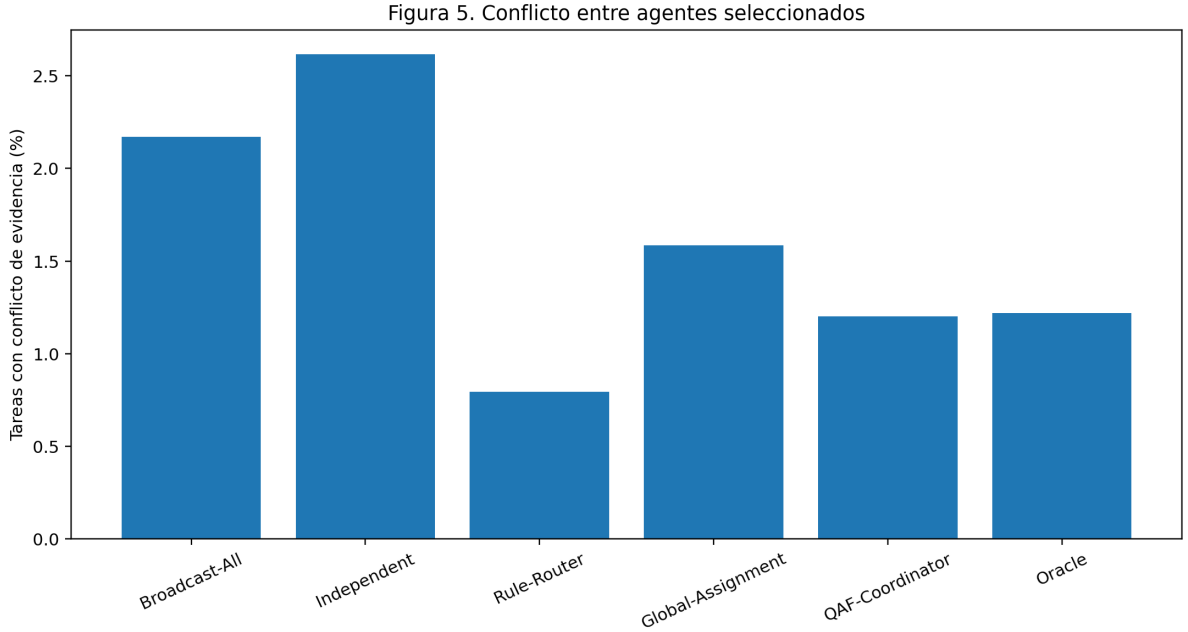


Figura 5: Conflicto entre agentes seleccionados.

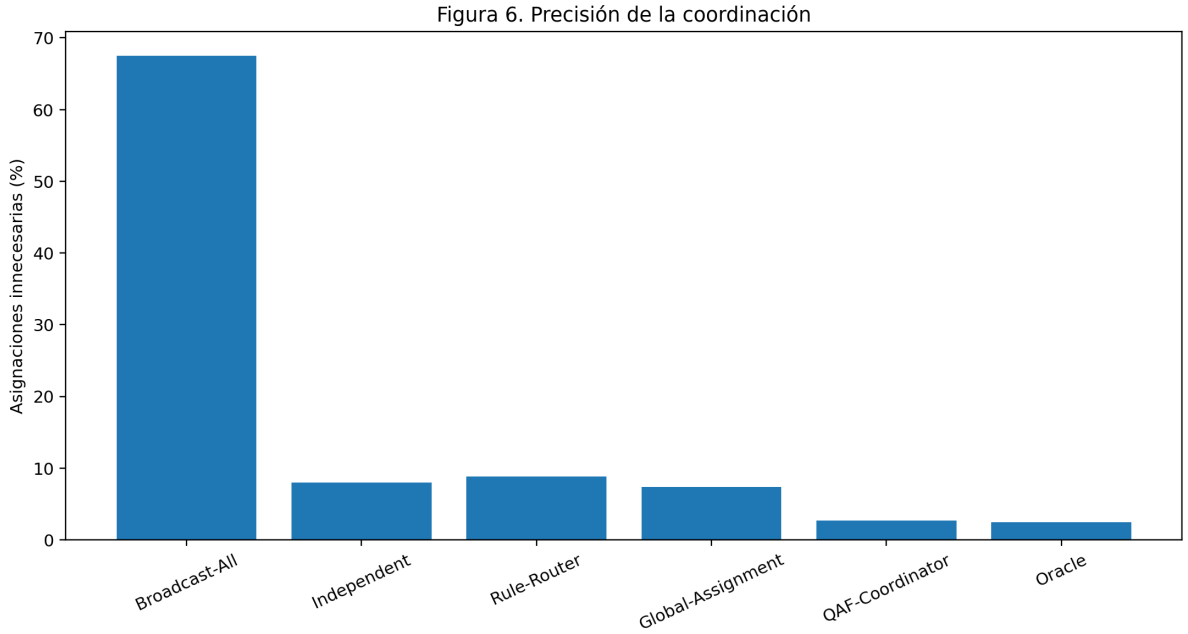


Figura 6: Asignaciones innecesarias.

7.3. Efecto de la incertidumbre

La Tabla 2 muestra que la ventaja de QAF es casi nula con 10 % de incertidumbre, aumenta con 35 % y llega a 17.88 % de mejora relativa con 60 %. Este patrón es consistente con H5: el mecanismo de diversidad y fusión contextual tiene poco que aportar cuando las observaciones ya son limpias, pero se vuelve más valioso cuando un consenso aparente puede estar producido por fuentes correlacionadas.

Cuadro 2: Éxito conjunto frente a incertidumbre.

Incertidumbre	Global (%)	QAF (%)	Mejora relativa
10 %	70.66	70.83	0.24 %
35 %	66.80	68.05	1.87 %
60 %	51.77	61.03	17.88 %

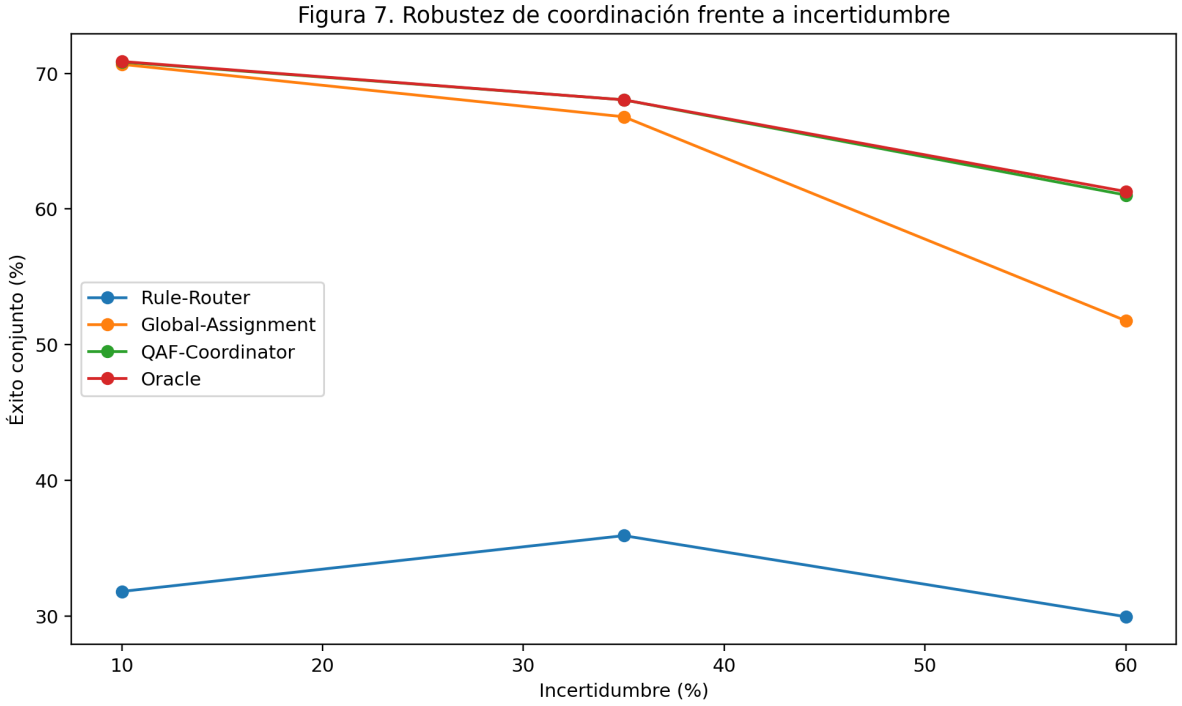


Figura 7: Robustez de coordinación frente a incertidumbre.

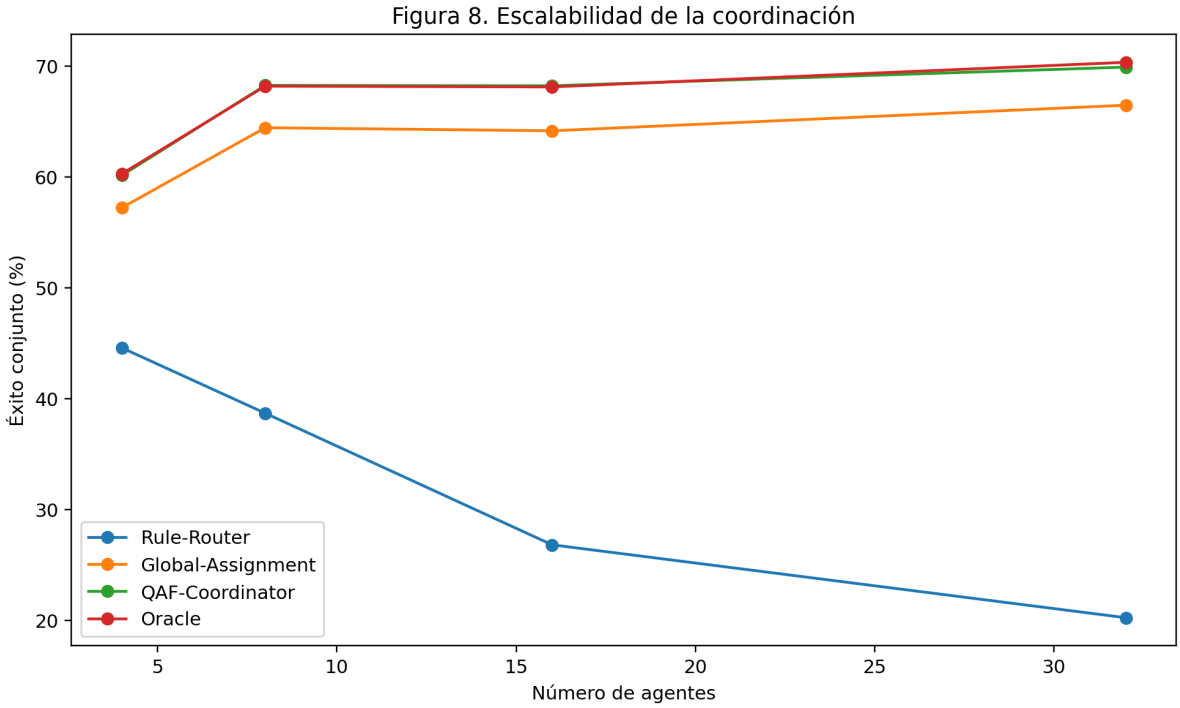


Figura 8: Escalabilidad con el número de agentes.

7.4. Frontera coordinación-comunicación

La Figura 9 resume la relación entre éxito y carga de comunicación. Broadcast-All maximiza difusión, pero sufre sobrecarga de capacidad y asignaciones redundantes. Global-Assignment mejora sustancialmente la calidad, pero consulta a todos los agentes. QAF se desplaza hacia una región de mayor éxito y menor comunicación, acercándose a la cota de enrutamiento del Oracle sin disponer de la verdad del escenario.

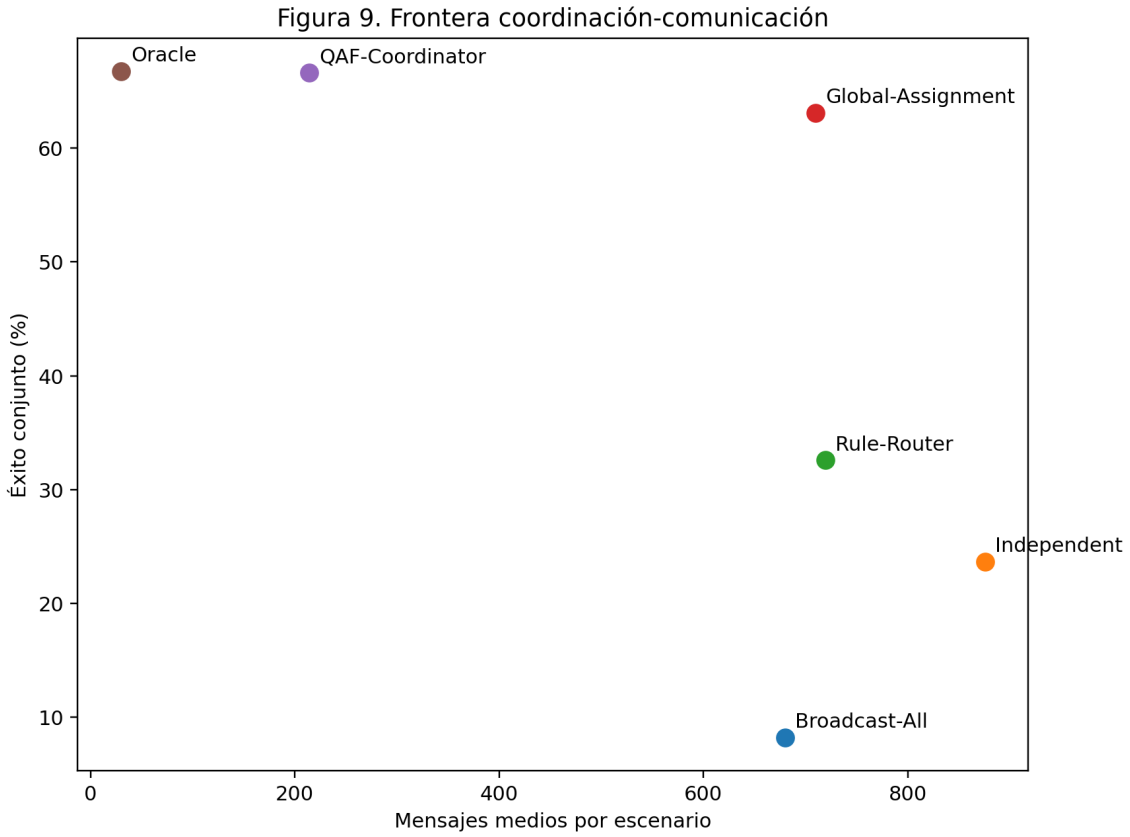


Figura 9: Frontera entre éxito conjunto y comunicación.

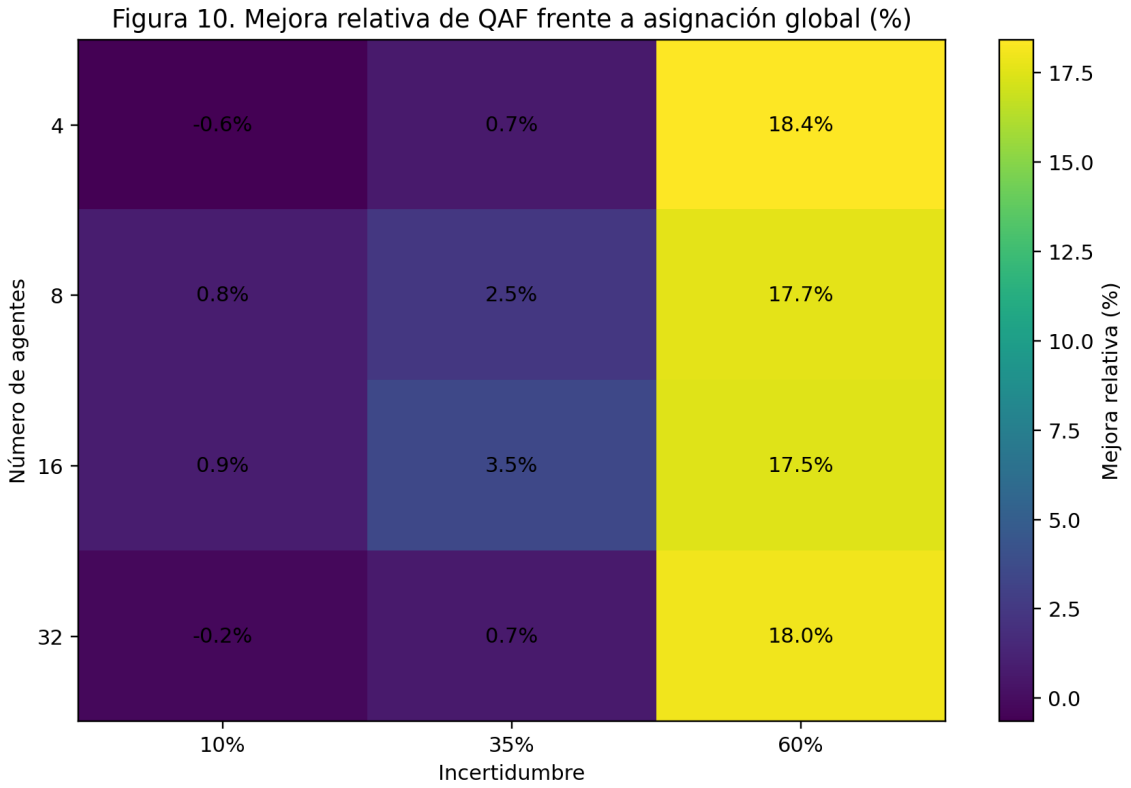


Figura 10: Mejora relativa de QAF frente a Global-Assignment por escala e incertidumbre.

8. Sabidurías derivadas del experimento

1. **Más comunicación no implica mejor coordinación.** Broadcast-All transmite mucha información pero genera sobreasignación y colisiones de capacidad. La coordinación requiere selección, no solo difusión.

2. **El consenso correlacionado puede ser engañoso.** Cuando muchas fuentes comparten el mismo sesgo, contarlas como evidencia independiente produce una falsa sensación de certeza. Balancear clústeres informacionales mejoró el desempeño con alta incertidumbre.

3. **La diversidad es una variable de coordinación.** Consultar dos fuentes informativamente distintas puede ser más útil que consultar diez fuentes redundantes.

4. **La metacognición puede actuar antes del razonamiento costoso.** QAF no necesita preguntar a todos para decidir a quién debe preguntar. Ese paso previo explica gran parte de la reducción de mensajes.

5. **La ventaja aparece donde el problema es difícil.** Con 10 % de incertidumbre QAF y Global-Assignment son equivalentes; con 60 % QAF obtiene una ventaja relativa de aproximadamente 17.88 %.

6. **La coordinación es un problema multiobjetivo.** El éxito debe leerse junto con mensajes, conflictos, cobertura, carga y latencia. Una arquitectura que reduce mensajes pero pierde cobertura no constituye una mejora.

7. **La asignación global sigue siendo un baseline fuerte.** El experimento no sugiere reemplazar la optimización clásica; QAF aporta principalmente en la construcción del contexto, la selección de evidencia y el routing previo a la asignación.

8. **Existe un intercambio cómputo-comunicación.** QAF requirió más tiempo local de coordinación (10.88 ms frente a 1.48 ms de Global-Assignment), pero redujo aproximadamente 69.87 % de los mensajes. En sistemas distribuidos, mover complejidad desde la red hacia cómputo local puede ser favorable, pero debe medirse end-to-end.

9. Interpretación científica

Los resultados apoyan preliminarmente la afirmación *"QAF puede mejorar la coordinación"*, pero la afirmación debe formularse con precisión: **en un benchmark sintético de agentes con fuentes correlacionadas, el proxy QAF de consulta selectiva, diversidad informacional y asignación global mejoró el éxito conjunto y redujo comunicación, conflictos y asignaciones innecesarias frente a una asignación global clásica**. Esto no equivale a demostrar que la formulación cuántica completa de QAF sea la causa del resultado.

El experimento revela una propuesta técnica más concreta para futuras validaciones: el valor potencial de QAF puede residir en representar *dependencia entre fuentes* y decidir qué información es marginalmente útil antes de gastar recursos en comunicación y razonamiento. En una futura implementación tensorial, estas dependencias deberían emerger del estado y de los operadores de acoplamiento, no de clústeres introducidos manualmente.

10. Limitaciones y condiciones de falsación

- El entorno es sintético; no contiene datos operacionales reales.
- El coordinador es un **proxy QAF**; no ejecuta MPS/QTT, TDVP, 1-RDM/2-RDM ni QDT completa.
- La señal contextual gruesa es simulada y parcialmente independiente; una implementación real debe demostrar que puede obtenerse sin introducir información privilegiada.
- La pertenencia a clústeres correlacionados es conocida por el experimento. En producción deberá estimarse a partir de historia, procedencia o estadística de errores.
- El Oracle conoce los requerimientos verdaderos y solo sirve como referencia de routing; no es un sistema implementable.
- El tiempo reportado mide el algoritmo local, no red, inferencia LLM, almacenamiento ni latencia de sensores.

La hipótesis de coordinación QAF debería considerarse refutada para esta clase de problemas si, al reemplazar el proxy por la implementación completa y usar datos controlados externos, no conserva una mejora de éxito frente a un baseline global fuerte o si la reducción de comunicación se obtiene a costa de menor cobertura y mayor error.

11. Próxima fase experimental

Se proponen cuatro extensiones: (1) estimar automáticamente correlaciones entre agentes a partir de historiales; (2) introducir pérdida, retraso y corrupción de mensajes; (3) reemplazar la señal contextual

simulada por un pipeline real de percepción/LLM; y (4) implementar el estado QAF MPS/TDVP y comparar si sus 1-RDM/2-RDM producen las mismas ventajas de diversidad y acoplamiento de forma endógena.

12. Conclusión

La prueba transforma una afirmación amplia en una proposición medible. En 360 escenarios pareados, el coordinador QAF alcanzó 66.64 % de éxito frente a 63.08 % del baseline global, redujo la comunicación en 69.87 % y disminuyó los conflictos en 24.11 %. La ventaja aumentó claramente con la incertidumbre, llegando a 17.88 % de mejora relativa a 60 % de ruido. El hallazgo central no es que QAF "coordine mejor" en cualquier circunstancia, sino que **la selección metacognitiva de fuentes, el control de evidencia correlacionada y la asignación global pueden mejorar simultáneamente calidad y eficiencia de coordinación cuando la información deja de ser limpia e independiente**. La siguiente etapa debe comprobar si el QAF matemático completo reproduce esa propiedad sin depender de supuestos introducidos manualmente.

Referencias

[1] Stone, P., & Veloso, M. (2000). Multiagent Systems: A Survey from a Machine Learning Perspective. *Autonomous Robots*, 8(3), 345–383.

[2] Gerkey, B. P., & Matarić, M. J. (2004). A formal analysis and taxonomy of task allocation in multi-robot systems. *International Journal of Robotics Research*, 23(9), 939–954. DOI: 10.1177/0278364904045564.

[3] Oliehoek, F. A., & Amato, C. (2016). *A Concise Introduction to Decentralized POMDPs*. Springer. DOI: 10.1007/978-3-319-28929-8.

[4] Sukhbaatar, S., Szlam, A., & Fergus, R. (2016). Learning Multiagent Communication with Backpropagation. *NeurIPS 29*, 2244–2252.

[5] Foerster, J. N., Assael, Y. M., de Freitas, N., & Whiteson, S. (2016). Learning to Communicate with Deep Multi-Agent Reinforcement Learning. *NeurIPS 29*.

[6] Das, A., Gervet, T., Romoff, J., Batra, D., Parikh, D., Rabbat, M., & Pineau, J. (2019). TarMAC: Targeted Multi-Agent Communication. *ICML*, PMLR 97, 1538–1546.

[7] Rojas Rodríguez, C. D. (2026). *Arquitectura quantum-like multiagente para apoyo a decisiones críticas en entornos de defensa bajo incertidumbre*. UNIR / ParkiApp Research.

[8] Bancells, A. (2026). *Quantum Analysis Framework (QAF): dinámica operacional no conmutativa y compilación QUBO asíncrona para la toma de decisiones complejas bajo incertidumbre*. Propuesta técnica ejecutiva.

A. Código Python reproducible

El siguiente código corresponde exactamente al benchmark ejecutado para producir los resultados y las diez figuras de este manuscrito.

```
"""
QAF Coordination Benchmark
Author: Juan Manuel Goenaga
Purpose: Experimental validation of the claim that a QAF-inspired
metacognitive/contextual coordinator can improve multi-agent coordination
under uncertainty by reducing communication, conflicts, unnecessary
assignments, and missed capability coverage while preserving or improving
joint task success.

IMPORTANT:
This is a reproducible QAF-inspired coordination proxy, not a full implementation
of the MPS/TDVP/1-RDM/2-RDM engine described in the technical QAF paper.
The experiment isolates three architectural ideas:
  1) selective/metacognitive querying,
  2) diversity-aware contextual fusion of correlated sources,
  3) capacity-aware global assignment.

The domain is intentionally abstract: tasks, agents, capabilities, and noisy
observations. It does not implement real-world targeting or weapons control.
"""

from pathlib import Path
import math, time, platform, sys
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import linear_sum_assignment
from scipy.stats import ttest_rel

SEED_BASE = 20260919
AGENT_COUNTS = [4, 8, 16, 32]
UNCERTAINTY_LEVELS = [0.10, 0.35, 0.60]
REPETITIONS = 30
CAPABILITIES = 4

BASE = Path(__file__).resolve().parent
RESULTS = BASE / "results"
FIGURES = BASE / "figures"
RESULTS.mkdir(exist_ok=True)
FIGURES.mkdir(exist_ok=True)

def generate_scenario(agent_count, uncertainty, seed):
    rng = np.random.default_rng(seed)
    A = agent_count
    C = CAPABILITIES
    T = 2 * A

    primary = rng.integers(0, C, size=A)
    proficiency = np.full((A, C), 0.10)
    for a in range(A):
        proficiency[a, primary[a]] = rng.uniform(0.82, 1.00)
        secondary = rng.choice([c for c in range(C) if c != primary[a]])
        proficiency[a, secondary] = rng.uniform(0.30, 0.55)

    reliability = np.clip(rng.beta(8, 2, size=A), 0.55, 0.99)

    # Two information clusters. Cluster 0 is deliberately dominant and more correlated.
    if A < 8:
        clusters = np.array([0] * (A - 1) + [1])
        rng.shuffle(clusters)
    else:
        n0 = max(2, int(round(A * 0.72)))
        clusters = np.array([0] * n0 + [1] * (A - n0))
        rng.shuffle(clusters)

    # Each agent can execute at most two assignments in the batch.
    capacity = np.full(A, 2, dtype=int)
```

```
    true_required = np.zeros((T, C), dtype=int)
    for t in range(T):
        k = 1 if rng.random() < 0.72 else 2
        true_required[t, rng.choice(C, size=k, replace=False)] = 1

    true_risk = (rng.random(T) < 0.55).astype(int)
    urgency = rng.uniform(0.45, 1.00, size=T)

    # Correlated observation errors create a realistic coordination failure mode:
    # a large information cluster can agree with itself and still be wrong.
    shared_capability_bias = np.zeros((2, T, C))
    shared_risk_bias = np.zeros((2, T))
    shared_capability_bias[0] = rng.normal(0, uncertainty * 0.90, size=(T, C))
    shared_capability_bias[1] = rng.normal(0, uncertainty * 0.35, size=(T, C))
    shared_risk_bias[0] = rng.normal(0, uncertainty * 0.90, size=T)
    shared_risk_bias[1] = rng.normal(0, uncertainty * 0.35, size=T)

    obs_capability = np.zeros((A, T, C))
    obs_risk = np.zeros((A, T))

    for a in range(A):
        individual_sigma = uncertainty * (1.10 - reliability[a]) + 0.035
        obs_capability[a] = np.clip(
            true_required
            + shared_capability_bias[clusters[a]]
            + rng.normal(0, individual_sigma, size=(T, C)),
            0, 1
        )
        obs_risk[a] = np.clip(
            true_risk
            + shared_risk_bias[clusters[a]]
            + rng.normal(0, individual_sigma, size=T),
            0, 1
        )

    # Coarse independent contextual signal available before deep agent consultation.
    coarse_capability = np.clip(
        true_required + rng.normal(0, uncertainty * 0.42 + 0.04, size=(T, C)),
        0, 1
    )
    coarse_risk = np.clip(
        true_risk + rng.normal(0, uncertainty * 0.42 + 0.04, size=T),
        0, 1
    )

    return {
        "A": A, "C": C, "T": T,
        "primary": primary,
        "proficiency": proficiency,
        "reliability": reliability,
        "clusters": clusters,
        "capacity": capacity,
        "true_required": true_required,
        "true_risk": true_risk,
        "urgency": urgency,
        "obs_capability": obs_capability,
        "obs_risk": obs_risk,
        "coarse_capability": coarse_capability,
        "coarse_risk": coarse_risk,
    }

def classical_fusion(s):
    cap = s["obs_capability"].mean(axis=0)
    risk = s["obs_risk"].mean(axis=0)
    cap_var = s["obs_capability"].var(axis=0)
    risk_var = s["obs_risk"].var(axis=0)
    return cap, risk, cap_var, risk_var

def assignments_broadcast_all(s):
    assignments = []
```

```
    for a in range(s["A"]):
        primary = int(np.argmax(s["proficiency"][a]))
        for t in range(s["T"]):
            assignments.append((a, t, primary))
    return assignments

def assignments_independent(s):
    assignments = []
    for a in range(s["A"]):
        for t in range(s["T"]):
            c = int(np.argmax(s["proficiency"][a] * s["obs_capability"][a, t]))
            activation = (
                s["obs_capability"][a, t, c]
                * s["reliability"][a]
                * s["proficiency"][a, c]
            )
            if activation > 0.50 and s["proficiency"][a, c] > 0.30:
                assignments.append((a, t, c))
    return assignments

def assignments_rule_router(s):
    fused, _, _, _ = classical_fusion(s)
    assignments = []
    for t in range(s["T"]):
        predicted = np.where(fused[t] >= 0.55)[0].tolist()
        if not predicted:
            predicted = [int(np.argmax(fused[t]))]
        predicted = sorted(predicted, key=lambda c: fused[t, c], reverse=True)[:2]
        for c in predicted:
            score = (
                s["obs_capability"][:, t, c]
                * s["proficiency"][:, c]
                * s["reliability"]
            )
            a = int(np.argmax(score))
            assignments.append((a, t, int(c)))
    return assignments

def assignments_global_classical(s):
    """
    Exact capacity-aware linear assignment using the Hungarian algorithm.
    It uses all agent observations and simple mean fusion. This is a strong
    centralized classical coordination baseline.
    """
    fused, _, _, _ = classical_fusion(s)
    slots = []
    for t in range(s["T"]):
        predicted = np.where(fused[t] >= 0.50)[0].tolist()
        if not predicted:
            predicted = [int(np.argmax(fused[t]))]
        predicted = sorted(predicted, key=lambda c: fused[t, c], reverse=True)[:2]
        for c in predicted:
            slots.append((t, int(c)))

    copies = [a for a in range(s["A"]) for _ in range(int(s["capacity"][a]))]
    if not slots:
        return []

    cost = np.zeros((len(copies), len(slots)))
    for i, a in enumerate(copies):
        for j, (t, c) in enumerate(slots):
            local = s["obs_capability"][a, t, c]
            score = (
                (0.75 * fused[t, c] + 0.25 * local)
                * s["proficiency"][a, c]
                * s["reliability"][a]
                * s["urgency"][t]
            )
            cost[i, j] = -score
```

```
    rows, cols = linear_sum_assignment(cost)
    assignments = []
    used_agent_task = set()
    for i, j in zip(rows, cols):
        a = copies[i]
        t, c = slots[j]
        if (a, t) in used_agent_task:
            continue
        used_agent_task.add((a, t))
        assignments.append((a, t, c))
    return assignments

def assignments_qaf_coordinator(s):
    """
    QAF-inspired coordinator proxy:
      - uses coarse context to decide which agents are worth consulting;
      - deliberately queries diverse information clusters;
      - fuses evidence after correcting for correlated-majority effects;
      - performs a global capacity-aware assignment.

    Returns assignments and the set of agent-task consultations.
    """
    A, T, C = s["A"], s["T"], s["C"]
    queried = set()

    # 1. Metacognitive selective query planning.
    for t in range(T):
        caps = np.where(s["coarse_capability"][t] >= 0.45)[0].tolist()
        if not caps:
            caps = [int(np.argmax(s["coarse_capability"][t]))]
        caps = sorted(caps, key=lambda c: s["coarse_capability"][t, c], reverse=True)[:2]

        for c in caps:
            # One high-quality source from each information cluster.
            for g in np.unique(s["clusters"]):
                idx = np.where(s["clusters"] == g)[0]
                score = s["reliability"][idx] * s["proficiency"][idx, c]
                a = int(idx[np.argmax(score)])
                queried.add((a, t))

    # 2. Diversity-aware contextual fusion.
    fused_cap = np.zeros((T, C))
    fused_risk = np.zeros(T)
    disagreement_cap = np.zeros((T, C))
    disagreement_risk = np.zeros(T)

    for t in range(T):
        queried_agents = [a for a, tt in queried if tt == t]
        group_caps = []
        group_risks = []

        for g in np.unique(s["clusters"]):
            idx = [a for a in queried_agents if s["clusters"][a] == g]
            if not idx:
                continue
            w = s["reliability"][idx] / s["reliability"][idx].sum()
            group_caps.append(np.tensordot(w, s["obs_capability"][idx, t, :], axes=(0, 0)))
            group_risks.append(float(np.dot(w, s["obs_risk"][idx, t])))

        if group_caps:
            gc = np.stack(group_caps)
            gr = np.array(group_risks)
            robust_cap = gc.mean(axis=0)
            robust_risk = gr.mean()
            disagreement_cap[t] = gc.var(axis=0)
            disagreement_risk[t] = gr.var()
        else:
            robust_cap = s["coarse_capability"][t]
            robust_risk = s["coarse_risk"][t]
```

```
        fused_cap[t] = 0.35 * s["coarse_capability"][t] + 0.65 * robust_cap
        fused_risk[t] = 0.35 * s["coarse_risk"][t] + 0.65 * robust_risk

    # 3. Global capacity-aware allocation.
    slots = []
    for t in range(T):
        predicted = np.where(fused_cap[t] >= 0.50)[0].tolist()
        if not predicted:
            predicted = [int(np.argmax(fused_cap[t]))]
        predicted = sorted(predicted, key=lambda c: fused_cap[t, c], reverse=True)[:2]
        for c in predicted:
            slots.append((t, int(c)))

    copies = [a for a in range(A) for _ in range(int(s["capacity"][a]))]
    if not slots:
        return [], queried

    cost = np.zeros((len(copies), len(slots)))
    for i, a in enumerate(copies):
        for j, (t, c) in enumerate(slots):
            if (a, t) in queried:
                local = s["obs_capability"][a, t, c]
            else:
                local = s["coarse_capability"][t, c]

            score = (
                (0.65 * fused_cap[t, c] + 0.20 * local + 0.15 * s["reliability"][a])
                * s["proficiency"][a, c]
                * s["urgency"][t]
            )

            # Small preference for agents whose evidence was explicitly consulted.
            if (a, t) in queried:
                score += 0.03

            # Under contradiction, prefer a capable member of the smaller cluster
            # to reduce correlated-majority failure.
            cluster_size = np.sum(s["clusters"] == s["clusters"][a])
            score += (
                0.10
                * np.sqrt(max(disagreement_cap[t, c], 0))
                * (A / cluster_size)
                * s["proficiency"][a, c]
            )
            cost[i, j] = -score

    rows, cols = linear_sum_assignment(cost)
    assignments = []
    used_agent_task = set()
    for i, j in zip(rows, cols):
        a = copies[i]
        t, c = slots[j]
        if (a, t) in used_agent_task:
            continue
        used_agent_task.add((a, t))
        assignments.append((a, t, c))

    return assignments, queried

def assignments_oracle(s):
    slots = []
    for t in range(s["T"]):
        for c in np.where(s["true_required"][t] == 1)[0]:
            slots.append((t, int(c)))

    copies = [a for a in range(s["A"]) for _ in range(int(s["capacity"][a]))]
    if not slots:
        return []

    cost = np.zeros((len(copies), len(slots)))
```

```
    for i, a in enumerate(copies):
        for j, (t, c) in enumerate(slots):
            cost[i, j] = -(
                s["proficiency"][a, c]
                * s["reliability"][a]
                * s["urgency"][t]
            )

    rows, cols = linear_sum_assignment(cost)
    assignments = []
    used_agent_task = set()
    for i, j in zip(rows, cols):
        a = copies[i]
        t, c = slots[j]
        if (a, t) in used_agent_task:
            continue
        used_agent_task.add((a, t))
        assignments.append((a, t, c))
    return assignments

def enforce_capacity(s, assignments):
    by_agent = {a: [] for a in range(s["A"])}
    for item in assignments:
        a, t, c = item
        by_agent[a].append(item)

    kept = []
    dropped = 0
    for a, items in by_agent.items():
        items = sorted(items, key=lambda it: s["urgency"][it[1]], reverse=True)
        cap = int(s["capacity"][a])
        kept.extend(items[:cap])
        dropped += max(0, len(items) - cap)
    return kept, dropped

def evaluate(s, assignments, method, queried=None):
    original_n = len(assignments)
    assignments, dropped = enforce_capacity(s, assignments)

    by_task = [[] for _ in range(s["T"])]
    loads = np.zeros(s["A"], dtype=int)
    for a, t, c in assignments:
        by_task[t].append((a, c))
        loads[a] += 1

    coverage_values = []
    decision_values = []
    unnecessary = 0
    duplicate = 0
    conflicts = 0
    route_complete = 0
    task_success = 0

    for t, team in enumerate(by_task):
        required = set(np.where(s["true_required"][t] == 1)[0].tolist())
        covered = set()
        cap_counts = {}
        votes = []
        weights = []

        for a, c in team:
            if c in required and s["proficiency"][a, c] > 0.30:
                covered.add(c)
            else:
                unnecessary += 1

            cap_counts[c] = cap_counts.get(c, 0) + 1
            votes.append(float(s["obs_risk"][a, t] >= 0.50))
            weights.append(s["reliability"][a])
```

```
        duplicate += sum(max(0, n - 1) for n in cap_counts.values())

        coverage = len(covered) / len(required) if required else 1.0
        coverage_values.append(coverage)

        complete = required.issubset(covered)
        route_complete += int(complete)

        if votes:
            prediction = np.average(np.array(votes), weights=np.array(weights)) >= 0.50
            correct = int(prediction == bool(s["true_risk"][t]))
            decision_values.append(correct)
            if len(set(votes)) > 1:
                conflicts += 1
        else:
            correct = 0
            decision_values.append(0)

        if complete and correct:
            task_success += 1

    if method == "Broadcast-All":
        messages = s["A"] * s["T"]
        queries = s["A"] * s["T"]
    elif method in ("Independent", "Rule-Router", "Global-Assignment"):
        messages = s["A"] * s["T"] + original_n
        queries = s["A"] * s["T"]
    elif method == "QAF-Coordinator":
        q = len(queried or [])
        # query request + response + task dispatch + assignment messages
        messages = 2 * q + s["T"] + original_n
        queries = q
    elif method == "Oracle":
        messages = original_n
        queries = 0
    else:
        messages = original_n
        queries = 0

    return {
        "task_success": task_success / s["T"],
        "role_coverage": float(np.mean(coverage_values)),
        "decision_accuracy": float(np.mean(decision_values)),
        "route_complete": route_complete / s["T"],
        "conflict_rate": conflicts / s["T"],
        "unnecessary_rate": unnecessary / max(1, len(assignments)),
        "duplicate_rate": duplicate / max(1, len(assignments)),
        "capacity_drop_rate": dropped / max(1, original_n),
        "messages": messages,
        "queries": queries,
        "assignments": len(assignments),
        "load_cv": float(np.std(loads) / (np.mean(loads) + 1e-9)),
    }

def run():
    rows = []
    methods = [
        ("Broadcast-All", assignments_broadcast_all),
        ("Independent", assignments_independent),
        ("Rule-Router", assignments_rule_router),
        ("Global-Assignment", assignments_global_classical),
        ("Oracle", assignments_oracle),
    ]

    for A in AGENT_COUNTS:
        for uncertainty in UNCERTAINTY_LEVELS:
            for rep in range(REPETITIONS):
                seed = SEED_BASE + A * 1000 + int(uncertainty * 100) * 10 + rep
                scenario = generate_scenario(A, uncertainty, seed)
```

```
            for name, fn in methods:
                t0 = time.perf_counter()
                assignments = fn(scenario)
                runtime_ms = (time.perf_counter() - t0) * 1000
                metrics = evaluate(scenario, assignments, name)
                rows.append({
                    "agents": A,
                    "uncertainty": uncertainty,
                    "rep": rep,
                    "seed": seed,
                    "method": name,
                    "runtime_ms": runtime_ms,
                    **metrics,
                })

            t0 = time.perf_counter()
            assignments, queried = assignments_qaf_coordinator(scenario)
            runtime_ms = (time.perf_counter() - t0) * 1000
            metrics = evaluate(scenario, assignments, "QAF-Coordinator", queried)
            rows.append({
                "agents": A,
                "uncertainty": uncertainty,
                "rep": rep,
                "seed": seed,
                "method": "QAF-Coordinator",
                "runtime_ms": runtime_ms,
                **metrics,
            })

df = pd.DataFrame(rows)
df.to_csv(RESULTS / "raw_results.csv", index=False)

summary_cols = [
    "task_success", "role_coverage", "decision_accuracy", "route_complete",
    "conflict_rate", "unnecessary_rate", "duplicate_rate",
    "capacity_drop_rate", "messages", "queries", "assignments",
    "load_cv", "runtime_ms"
]

summary = df.groupby("method")[summary_cols].agg(["mean", "std"])
summary.to_csv(RESULTS / "summary.csv")

uncertainty_summary = (
    df.groupby(["uncertainty", "method"])[summary_cols]
    .mean()
    .reset_index()
)
uncertainty_summary.to_csv(RESULTS / "uncertainty_summary.csv", index=False)

scaling_summary = (
    df.groupby(["agents", "method"])[summary_cols]
    .mean()
    .reset_index()
)
scaling_summary.to_csv(RESULTS / "agent_scaling_summary.csv", index=False)

# Paired QAF vs Global tests
paired = (
    df[df["method"].isin(["Global-Assignment", "QAF-Coordinator"])]
    .pivot(index=["agents", "uncertainty", "rep"], columns="method", values="task_success")
    .dropna()
)
diff = paired["QAF-Coordinator"] - paired["Global-Assignment"]
test = ttest_rel(paired["QAF-Coordinator"], paired["Global-Assignment"])
mean_diff = float(diff.mean())
se = float(diff.std(ddof=1) / np.sqrt(len(diff)))
ci_low, ci_high = mean_diff - 1.96 * se, mean_diff + 1.96 * se

global_mean = float(df.loc[df["method"] == "Global-Assignment", "task_success"].mean())
qaf_mean = float(df.loc[df["method"] == "QAF-Coordinator", "task_success"].mean())
```

```
    global_msg = float(df.loc[df["method"] == "Global-Assignment", "messages"].mean())
    qaf_msg = float(df.loc[df["method"] == "QAF-Coordinator", "messages"].mean())
    global_conflict = float(df.loc[df["method"] == "Global-Assignment", "conflict_rate"].mean())
    qaf_conflict = float(df.loc[df["method"] == "QAF-Coordinator", "conflict_rate"].mean())
    global_unnecessary = float(df.loc[df["method"] == "Global-Assignment", "unnecessary_rate"].mean())
    qaf_unnecessary = float(df.loc[df["method"] == "QAF-Coordinator", "unnecessary_rate"].mean())

    stats = pd.DataFrame([{
        "n_paired_scenarios": len(diff),
        "qaf_task_success": qaf_mean,
        "global_task_success": global_mean,
        "relative_success_improvement_pct": (qaf_mean - global_mean) / global_mean * 100,
        "absolute_success_improvement_pp": (qaf_mean - global_mean) * 100,
        "paired_mean_difference": mean_diff,
        "paired_ci95_low": ci_low,
        "paired_ci95_high": ci_high,
        "paired_t_stat": float(test.statistic),
        "paired_p_value": float(test.pvalue),
        "message_reduction_pct": (global_msg - qaf_msg) / global_msg * 100,
        "conflict_reduction_pct": (global_conflict - qaf_conflict) / global_conflict * 100,
        "unnecessary_assignment_reduction_pct": (global_unnecessary - qaf_unnecessary) / global_unnecessary * 100,
    }])
    stats.to_csv(RESULTS / "qaf_vs_global_statistics.csv", index=False)

    create_figures(df)
    return df, stats

def savefig(fig, name):
    fig.tight_layout()
    fig.savefig(FIGURES / name, dpi=220, bbox_inches="tight")
    plt.close(fig)

def create_figures(df):
    order = ["Broadcast-All", "Independent", "Rule-Router", "Global-Assignment", "QAF-Coordinator", "Oracle"]
    overall = df.groupby("method").mean(numeric_only=True).reindex(order)

    # Figure 1: architecture diagram
    fig, ax = plt.subplots(figsize=(11, 6))
    ax.axis("off")
    boxes = [
        (0.06, 0.68, "Evento / tarea\n+ contexto grueso"),
        (0.28, 0.68, "Metacognicion QAF\na quien consultar?"),
        (0.50, 0.68, "Fuentes diversas\nconsulta selectiva"),
        (0.72, 0.68, "Fusion contextual\ncontradiccion / confianza"),
        (0.50, 0.25, "Asignacion global\nbajo capacidad"),
        (0.72, 0.25, "Equipo coordinado\n+ decision conjunta"),
    ]
    for x, y, label in boxes:
        ax.text(x, y, label, transform=ax.transAxes, ha="center", va="center",
                bbox=dict(boxstyle="round,pad=0.6", fill=False))
    arrows = [
        ((0.12,0.68),(0.22,0.68)), ((0.34,0.68),(0.44,0.68)),
        ((0.56,0.68),(0.66,0.68)), ((0.72,0.61),(0.56,0.34)),
        ((0.56,0.25),(0.66,0.25))
    ]
    for p1,p2 in arrows:
        ax.annotate("", xy=p2, xytext=p1, xycoords=ax.transAxes,
                    arrowprops=dict(arrowstyle="->"))
    ax.set_title("Figura 1. Arquitectura experimental del coordinador QAF")
    savefig(fig, "fig01_architecture.png")

    # Figure 2
    fig, ax = plt.subplots(figsize=(10, 5.5))
    ax.bar(overall.index, overall["task_success"] * 100)
    ax.set_ylabel("Exito conjunto de tarea (%)")
    ax.set_title("Figura 2. Exito conjunto de coordinacion")
    ax.tick_params(axis="x", rotation=25)
    savefig(fig, "fig02_task_success.png")

    # Figure 3
```

```
fig, ax = plt.subplots(figsize=(10, 5.5))
ax.bar(overall.index, overall["role_coverage"] * 100)
ax.set_ylabel("Cobertura de capacidades requeridas (%)")
ax.set_title("Figura 3. Cobertura operacional de capacidades")
ax.tick_params(axis="x", rotation=25)
savefig(fig, "fig03_role_coverage.png")

# Figure 4
fig, ax = plt.subplots(figsize=(10, 5.5))
ax.bar(overall.index, overall["messages"])
ax.set_ylabel("Mensajes medios por escenario")
ax.set_title("Figura 4. Sobrecarga de comunicacion")
ax.tick_params(axis="x", rotation=25)
savefig(fig, "fig04_messages.png")

# Figure 5
fig, ax = plt.subplots(figsize=(10, 5.5))
ax.bar(overall.index, overall["conflict_rate"] * 100)
ax.set_ylabel("Tareas con conflicto de evidencia (%)")
ax.set_title("Figura 5. Conflicto entre agentes seleccionados")
ax.tick_params(axis="x", rotation=25)
savefig(fig, "fig05_conflict_rate.png")

# Figure 6
fig, ax = plt.subplots(figsize=(10, 5.5))
ax.bar(overall.index, overall["unnecessary_rate"] * 100)
ax.set_ylabel("Asignaciones innecesarias (%)")
ax.set_title("Figura 6. Precision de la coordinacion")
ax.tick_params(axis="x", rotation=25)
savefig(fig, "fig06_unnecessary.png")

# Figure 7
fig, ax = plt.subplots(figsize=(9, 5.5))
focus = df[df["method"].isin(["Rule-Router", "Global-Assignment", "QAF-Coordinator", "Oracle"])]
grp = focus.groupby(["uncertainty", "method"])["task_success"].mean().reset_index()
for method in ["Rule-Router", "Global-Assignment", "QAF-Coordinator", "Oracle"]:
    sub = grp[grp["method"] == method]
    ax.plot(sub["uncertainty"] * 100, sub["task_success"] * 100, marker="o", label=method)
ax.set_xlabel("Incertidumbre (%)")
ax.set_ylabel("Exito conjunto (%)")
ax.set_title("Figura 7. Robustez de coordinacion frente a incertidumbre")
ax.legend()
savefig(fig, "fig07_uncertainty.png")

# Figure 8
fig, ax = plt.subplots(figsize=(9, 5.5))
grp = focus.groupby(["agents", "method"])["task_success"].mean().reset_index()
for method in ["Rule-Router", "Global-Assignment", "QAF-Coordinator", "Oracle"]:
    sub = grp[grp["method"] == method]
    ax.plot(sub["agents"], sub["task_success"] * 100, marker="o", label=method)
ax.set_xlabel("Numero de agentes")
ax.set_ylabel("Exito conjunto (%)")
ax.set_title("Figura 8. Escalabilidad de la coordinacion")
ax.legend()
savefig(fig, "fig08_agents_scaling.png")

# Figure 9
fig, ax = plt.subplots(figsize=(8, 6))
for method in order:
    row = overall.loc[method]
    ax.scatter(row["messages"], row["task_success"] * 100, s=70)
    ax.annotate(method, (row["messages"], row["task_success"] * 100), xytext=(5, 5),
                textcoords="offset points")
ax.set_xlabel("Mensajes medios por escenario")
ax.set_ylabel("Exito conjunto (%)")
ax.set_title("Figura 9. Frontera coordinacion-comunicacion")
savefig(fig, "fig09_pareto.png")

# Figure 10: relative QAF improvement heatmap
g = (
```

```
        df[df["method"].isin(["Global-Assignment", "QAF-Coordinator"])]
        .groupby(["agents", "uncertainty", "method"])["task_success"]
        .mean()
        .unstack("method")
        .reset_index()
    )
    g["improvement"] = (
        (g["QAF-Coordinator"] - g["Global-Assignment"])
        / g["Global-Assignment"]
        * 100
    )
    matrix = g.pivot(index="agents", columns="uncertainty", values="improvement")
    fig, ax = plt.subplots(figsize=(8, 5.5))
    im = ax.imshow(matrix.values, aspect="auto")
    ax.set_xticks(range(len(matrix.columns)))
    ax.set_xticklabels([f"{u*100:.0f}%" for u in matrix.columns])
    ax.set_yticks(range(len(matrix.index)))
    ax.set_yticklabels([str(a) for a in matrix.index])
    ax.set_xlabel("Incertidumbre")
    ax.set_ylabel("Numero de agentes")
    ax.set_title("Figura 10. Mejora relativa de QAF frente a asignacion global (%)")
    for i in range(matrix.shape[0]):
        for j in range(matrix.shape[1]):
            ax.text(j, i, f"{matrix.iloc[i,j]:.1f}%", ha="center", va="center")
    fig.colorbar(im, ax=ax, label="Mejora relativa (%)")
    savefig(fig, "fig10_heatmap.png")

if __name__ == "__main__":
    df, stats = run()
    print("Benchmark completado.")
    print(f"Escenarios unicos: {len(AGENT_COUNTS) * len(UNCERTAINTY_LEVELS) * REPETITIONS}")
    print(f"Ejecuciones de arquitectura: {len(df)}")
    print("\nResumen medio:")
    print(df.groupby("method")[[
        "task_success", "role_coverage", "decision_accuracy",
        "conflict_rate", "unnecessary_rate", "messages", "runtime_ms"
    ]].mean().sort_values("task_success", ascending=False).round(4))
    print("\nQAF vs Global-Assignment:")
    print(stats.T.to_string(header=False))
    print("\nEntorno:")
    print(platform.platform())
    print("Python", sys.version.split()[0])
```